
Code/A9_2.py

```
import matplotlib.pyplot as plt
import numpy as np
import warnings
from utils import *

def quasi_newton(f, x0, B0, rho=0.5, c1=1e-3, eps=1e-10, tau=1e-10, kmax=100, plot=True, display=True):
    """
    BFGS-basiertes Quasi-Newton-Verfahren zur Minimierung der Funktion f.

    Input:
        f: Funktion, wobei  $f(x) = (val, grad, hess)$ ,
        x0: Vektor (Startwert),
        B0: Matrix (Initiale Schätzung für die inverse Hesse-Matrix),
        rho, c1: Skalare (Armijo Parameter),
        tau, eps, kmax: Skalare (Abbruchkriterien nach Gill-Murray-Wright),
        plot: Boolescher Wert (gibt an, ob Daten zur Ploterstellung gespeichert werden sollen),
        display: Boolescher Wert (gibt an, ob Zwischenwerte aus den Iterationen in der Konsole ausgegeben

    Output:
        log: Dictionary vom Verlauf der Optimierung.
    """
    # Dictionary zum Abspeichern der Ergebnisse.
    log = {
        'x0': x0,                # Startwert
        'xqn': None,             # Lösung des CG-Verfahrens
        'kqn': None,             # Anzahl benötigter Iterationen
        'x_list': [],            # Liste der Iterierten
        'val_list': [],           # Liste des Funktionswerts in den Iterierten
        'norm_grad_list': [],     # Liste der Norm des Gradienten in den Iterierten
    }

    # TODO: Aufgabenteil 1. Quasi-Newton-Verfahren implementieren.
    # BEGIN SOLUTION
    Bk = B0
    xk = x0
    valk, gradfxk, _ = f(xk)
    pk = np.dot(Bk, - gradfxk)
    alphak = armijo(f, pk, xk, rho, c1)
    xkplus1 = xk + alphak * pk
    valkplus1, gradfxkplus1, _ = f(xkplus1)
    k = 0

    #speichere alle Einträge im log
    if plot:
```

```

log['x_list'].append(xk)
log['val_list'].append(alk)
log['norm_grad_list'].append(np.linalg.norm(gradfxk))
log['x_list'].append(xkplus1)
log['val_list'].append(alkplus1)
log['norm_grad_list'].append(np.linalg.norm(gradfxkplus1))

if (display):
    print(f'Ausgabe der Werte in der {k}-ten Iteration:')
    print(f'alpha^k ist: {alphak}')
    print(f'x^k ist: {xk}')
    print(f'f(x^k) ist: {alkplus1}')

#a) Überprüfe Abbruchkriterien
while (not gill_murray_wright(xk, xkplus1, alk, alkplus1, gradfxkplus1, k, eps, tau, kmax)):

    #e) Berechne s^k, y^k und überprüfe, ob deren Produkt >0 ist
    sk = xkplus1 - xk
    yk = gradfxkplus1 - gradfxk
    if(np.dot(yk.T, sk) <= 0):
        warnung = f"s^k * y^k <= 0 in Iteration: {k}"
        warnings.warn(warnung)
        break

    #f) Berechne B_(k+1)
    rohk = 1/(np.dot(yk, sk))
    Bk = (np.eye(len(yk)) - rohk * np.dot(sk,yk.T)) @ Bk @ (np.eye(len(yk)) - rohk * np.dot(yk,sk.T))

    k+=1

    # b) Berechne Suchrichtung
    pk = np.matmul(Bk, (-gradfxkplus1))

    # c) Bestimme Schrittweite, die die Armijo-Bed. erfüllt
    alphak = armijo(f, pk, xkplus1, rho, c1)

    # d) Bestimme neues x^k
    xk = xkplus1
    xkplus1 = xkplus1 + alphak * pk

    # Update Werte und Gradienten und berechne Wert und Gradienten an x^(k+1)
    alk = alkplus1
    gradfxk = gradfxkplus1
    alkplus1, gradfxkplus1, _ = f(xkplus1)

    # gebe alle Einträge aus, sofern display=True ist
    if (display):
        print(f'Ausgabe der Werte in der {k}-ten Iteration:')
        print(f'alpha^k ist: {alphak}')
        print(f'x^k ist: {xk}')
        print(f'f(x^k) ist: {alk}')

    # speichere alle Einträge im log
    if plot:
        log['x_list'].append(xkplus1)

```

```

        log['val_list'].append(alkplus1)
        log['norm_grad_list'].append(np.linalg.norm(gradfxkplus1))

    log['kqn'] = k
    log['xqn'] = xkplus1
    # END SOLUTION
    return log

if __name__ == '__main__':
    # TODO: Aufgabenteil 3. Test-Werte anlegen, Verfahren für tau = 1e-3 und tau = 1e-10 testen und Verlauf
    # BEGIN SOLUTION
    f = lambda x: rosenbrock(*x.T)

    #Für tau=1e-10
    print('-----Testen Verlauf für tau = 1e-10 und x0 = (-1.2, 1)-----')
    result_tau_big = quasi_newton(f, np.array([-1.2,1]), np.identity(2), rho = 0.5, c1 = 1e-3, eps = 1e-10)
    minimizer = result_tau_big['xqn']
    print(f'Der gefundene Minimierer ist: {minimizer}')

    #Für tau=1e-2
    print('-----Testen Verlauf für tau = 1e-2 und x0 = (-1.2, 1)-----')
    result_tau_small = quasi_newton(f, np.array([-1.2,1]), np.eye(2), rho = 0.5, c1 = 1e-3, eps = 1e-2, ta
    minimizer = result_tau_small['xqn']
    print(f'Der gefundene Minimierer ist: {minimizer}\n')

    #Plotte die Ergebnisse
    fig = plot_iteration_rosenbrock(result_tau_big, "Quasi-Newton mit tau = 1e-10")
    plt.show()

    fig = plot_iteration_rosenbrock(result_tau_small, "Quasi-Newton mit tau = 1e-2")
    plt.show()

    # END SOLUTION

    # TODO: Aufgabenteil 3. Verfahren für x0 = (0, -0.75) testen und Verlauf plotten
    # BEGIN SOLUTION

    #Für tau=1e-10
    print('-----Testen Verlauf für tau = 1e-10 und x0 = (0, -0.75)-----')
    result_tau_big1 = quasi_newton(f, np.array([0, -0.75]), np.eye(2), rho = 0.5, c1 = 1e-3, eps = 1e-10,
    minimizer = result_tau_big1['xqn']
    print(f'Der gefundene Minimierer ist: {minimizer}')

    #Für tau=1e-3
    print('-----Testen Verlauf für tau = 1e-2 und x0 = (0, -0.75)-----')
    result_tau_small1 = quasi_newton(f, np.array([0, -0.75]), np.eye(2), rho = 0.5, c1 = 1e-3, eps = 1e-2,
    minimizer = result_tau_small1['xqn']
    print(f'Der gefundene Minimierer ist: {minimizer}\n')

    #Plotte die Ergebnisse
    fig = plot_iteration_rosenbrock(result_tau_big1, "Quasi-Newton mit tau = 1e-10")
    plt.show()

```

```

fig = plot_iteration_rosenbrock(result_tau_small1, "Quasi-Newton mit tau = 1e-2")
plt.show()

# END SOLUTION

# TODO: Aufgabenteil 3. Diskussion
# BEGIN SOLUTION
print("Bei beiden Startwerten hat die Größe von Tau keinen Einfluss auf die Anzahl der Iterationen,\n"
      "da im ersten Fall der Minimierer jeweils nach 8 Iterationen gefunden wird und im zweiten Fall b"
      " die 100 Iteration durchlaufen werden. \n")
# END SOLUTION

```

#TERMINAL OUTPUT:

-----Testen Verlauf für tau = 1e-10 und x0 = (-1.2, 1)-----

Der gefundene Minimierer ist: [0.98490498 1.04488723]

-----Testen Verlauf für tau = 1e-2 und x0 = (-1.2, 1)-----

Der gefundene Minimierer ist: [0.98490498 1.04488723]

-----Testen Verlauf für tau = 1e-10 und x0 = (0, -0.75)-----

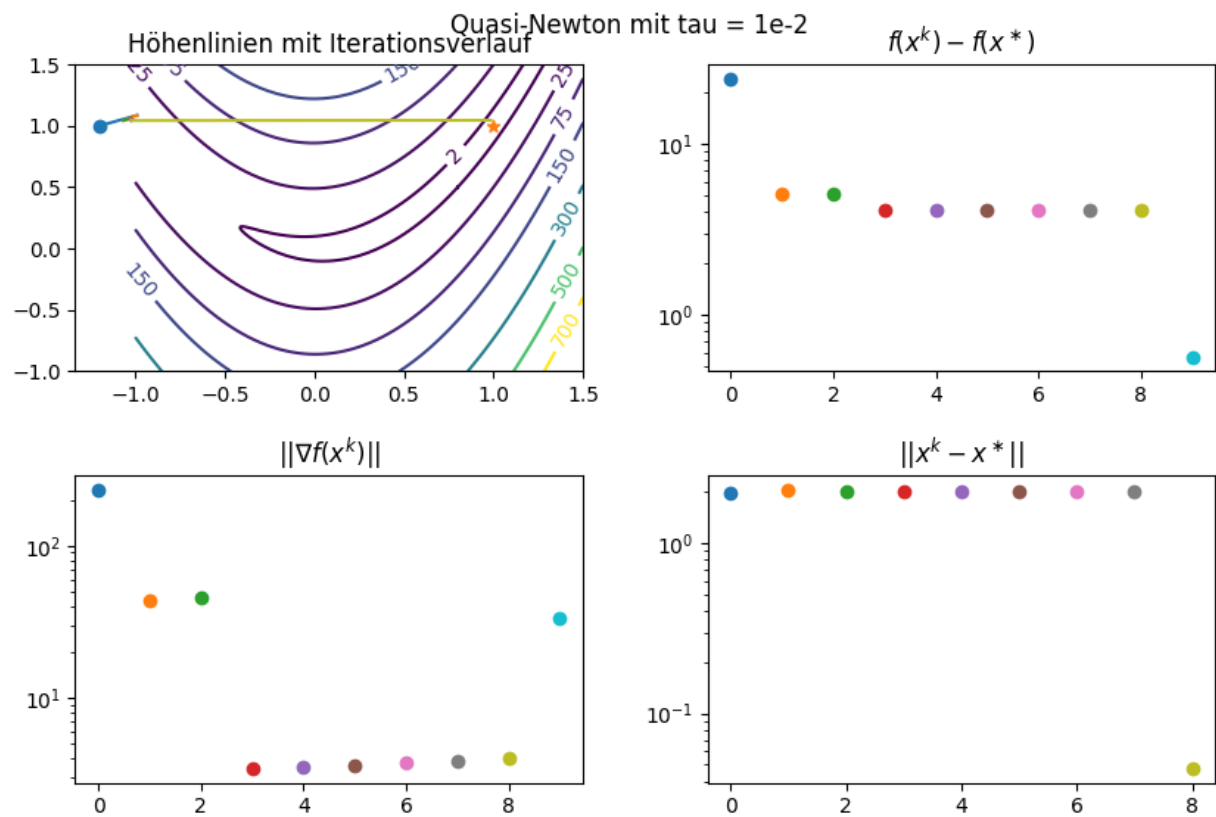
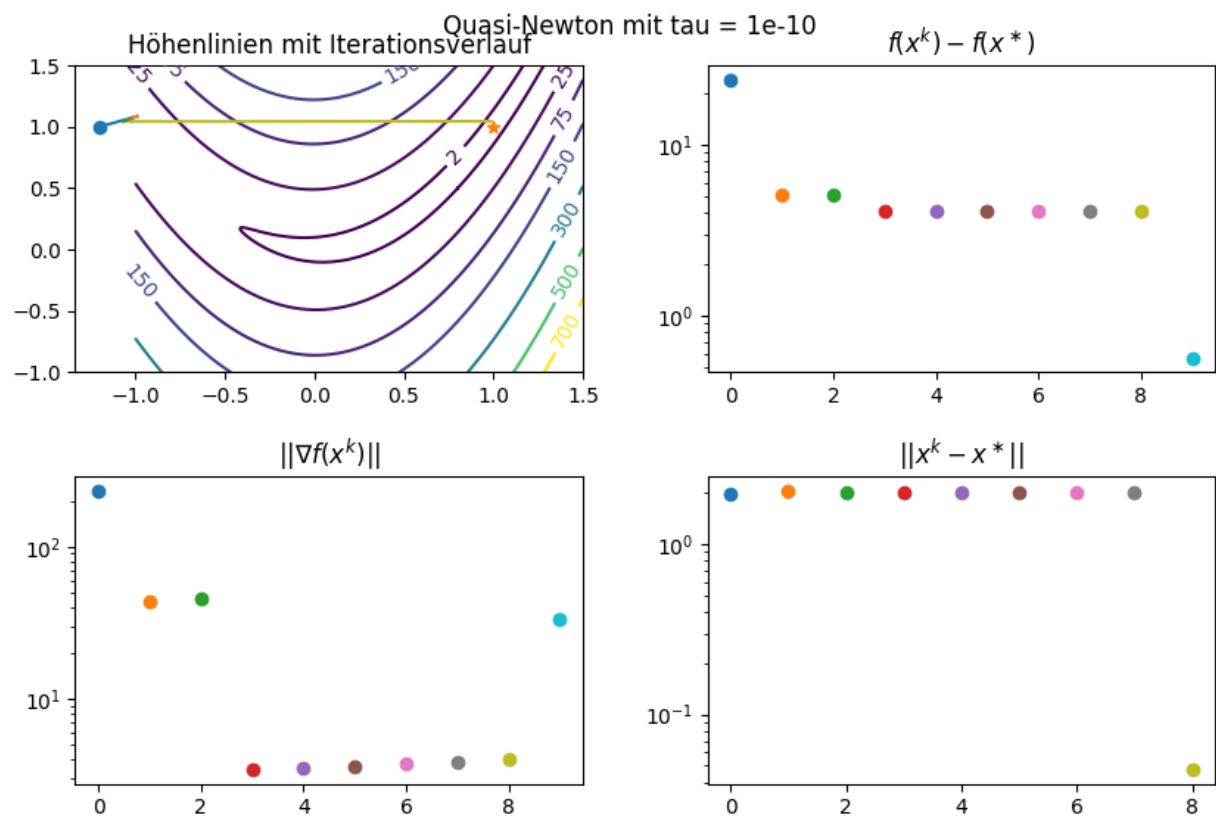
Der gefundene Minimierer ist: [1.06075643 1.14607317]

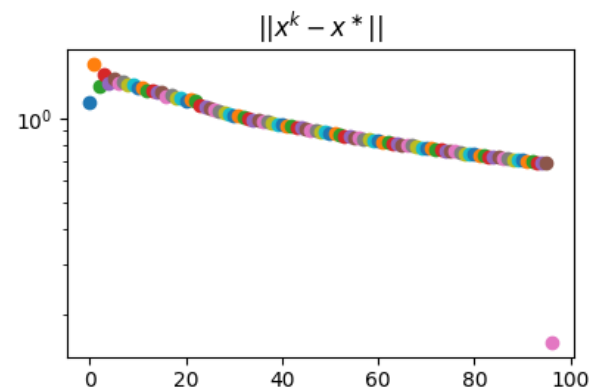
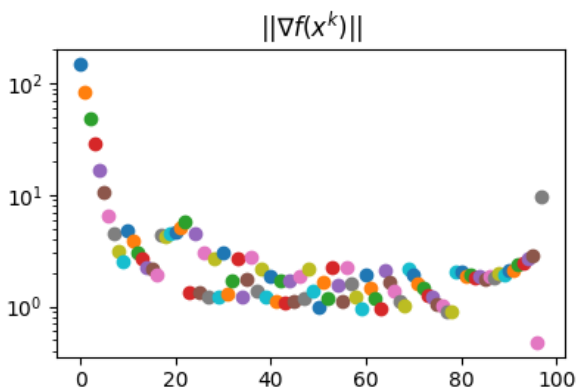
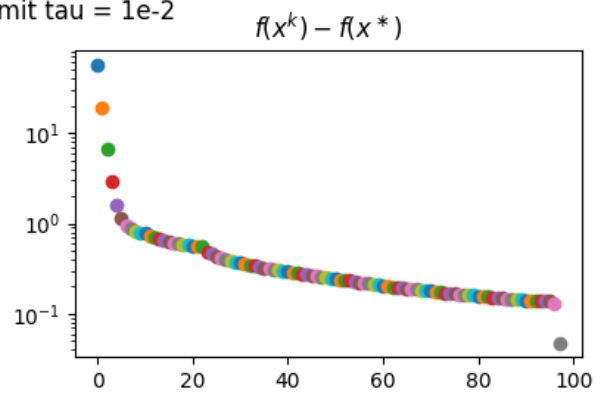
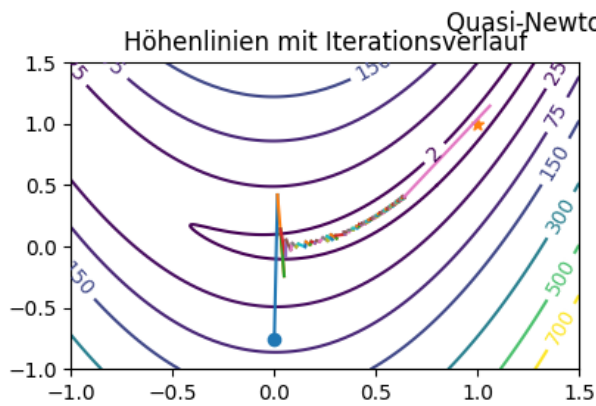
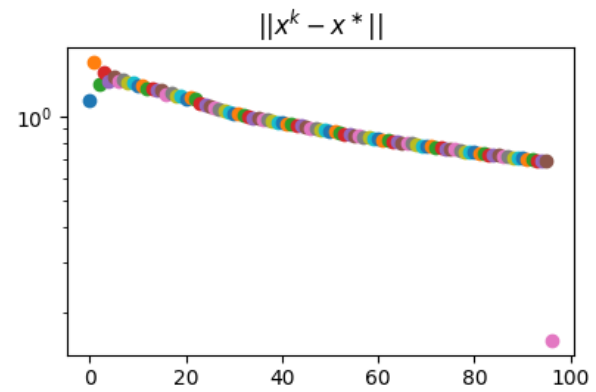
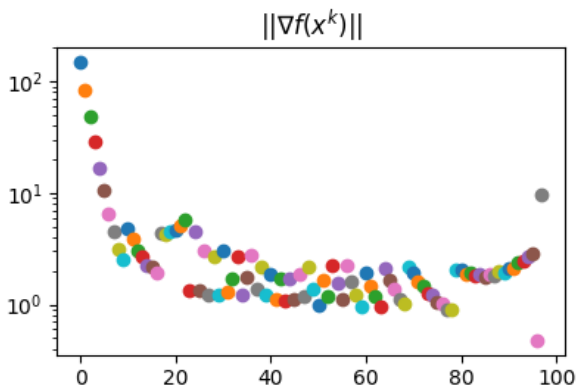
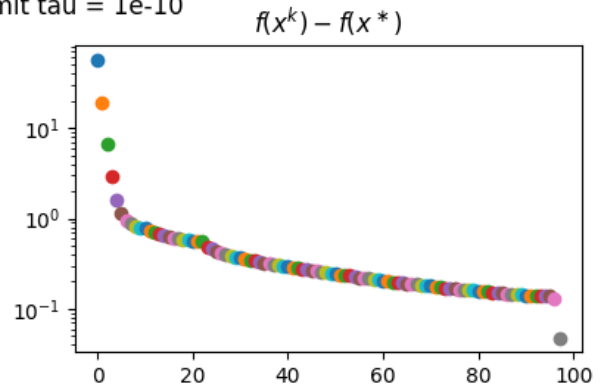
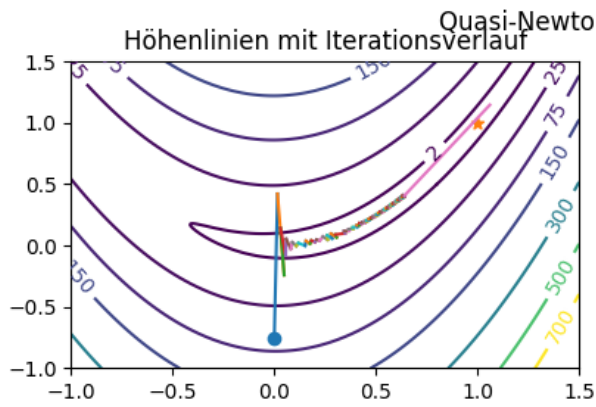
-----Testen Verlauf für tau = 1e-2 und x0 = (0, -0.75)-----

Der gefundene Minimierer ist: [1.06075643 1.14607317]

Bei beiden Startwerten hat die Größe von Tau keinen Einfluss auf die Anzahl der Iterationen,
da im ersten Fall der Minimierer jeweils nach 8 Iterationen gefunden wird und im zweiten Fall beide Male
die 100 Iteration durchlaufen werden.

#PLOTS:





Code/A9_3.py

```
import numpy as np
from skimage import io
from scipy.sparse import spdiags, vstack, kron
from scipy.sparse.linalg import spsolve
import matplotlib.pyplot as plt

#from A4.code.utils import plot_iteration_process
from A9_2 import quasi_newton

# weitere Module importieren
# BEGIN SOLUTION
...
# END SOLUTION

import os
class RigidRegistration:
    def __init__(self):
        self.ref_image = io.imread(os.path.join(os.path.split(__file__)[0], 'dataR.png')).astype(np.float64)
        self.tmp_image = io.imread(os.path.join(os.path.split(__file__)[0], 'dataT.png')).astype(np.float64)

        self.m = self.tmp_image.shape
        self.omega = np.array([0, self.m[0], 0, self.m[1]])
        self.h = (self.omega[1::2] - self.omega[0::2]) / np.array([self.m[1], self.m[0]])

        self.ref_grid = self.grid()
        self.tmp_grid = self.grid()

    def grid(self):
        # Get cell centered grid
        grid_xi = lambda i: np.array([self.omega[2*i] + (k+1/2)*self.h[i] for k in np.arange(self.m[i])])
        xi = list(reversed([grid_xi(i) for i in range(2)]))
        xc = list(np.meshgrid(*xi, indexing='ij'))
        xc = np.vstack([xc[i].flatten() for i in range(2)])
        return xc

    def deform_grid(self, grid, w):
        c = ((self.omega[1::2] - self.omega[0::2]) / 2).reshape(2, 1)
        theta, t = w[0], w[1:]
        R = np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])
        dR = np.array([[ -np.sin(theta), -np.cos(theta)], [np.cos(theta), -np.sin(theta)]])

        phi = R @ (grid - c) + c + t[:, np.newaxis]
        dphi = dR @ (self.tmp_grid - c)[np.newaxis, :, :]
        dphi = np.concatenate((dphi, np.tile(np.eye(2)[:, :, np.newaxis], (1, 1, dphi.shape[2]))), axis=0)
        return phi, dphi

    def interpolate(self, image, grid):
        def Bspline3(x):
            condition = lambda i: np.logical_and(i <= x, x < i+1)

            f = np.where(condition(0), (1/6) * (x**3), 0)
            f = np.where(condition(1), (1/6) * (-3 * x**3 + 12 * x**2 - 12 * x + 4), f)
            f = np.where(condition(2), (1/6) * (3 * x**3 - 24 * x**2 + 60 * x - 44), f)
```

```

f = np.where(condition(3), (1/6) * (4 - x)**3, f)

g = np.where(condition(0), (1/6) * (3 * x**2), 0)
g = np.where(condition(1), (1/6) * (-9 * x**2 + 24 * x - 12), g)
g = np.where(condition(2), (1/6) * (9 * x**2 - 48 * x + 60), g)
g = np.where(condition(3), (1/6) * -3 * (4 - x)**2, g)
return f, g

def paddedMatrix(dim):
    ones = np.ones(dim + 3)
    A = spdiags(data=[ones, 4*ones, ones], diags=[0, 1, 2], m=dim+1, n=dim+3)

    tmp_0 = spdiags(data=[6*ones, -12*ones, 6*ones], diags=[0, 1, 2], m=1, n=dim+3)
    tmp_m = spdiags(data=[6*ones, -12*ones, 6*ones], diags=[dim+0, dim+1, dim+2], m=1, n=dim+3)

    A_padded = vstack((tmp_0, A, tmp_m)) / 6
    return A_padded

def naturalSpline3():
    A_padded = kron(paddedMatrix(self.m[0] - 1), paddedMatrix(self.m[1] - 1))
    data_padded = np.pad(image, (1, 1), 'constant', constant_values=(0, 0))
    C = spsolve(A_padded.tocsc(), data_padded.flatten()).reshape(self.m[0]+2, self.m[1]+2)
    return C

# Redistribute Grid: Map grid points from [h/3, omega-h/3] to [h/3, m[i]-h/3]
points_x, points_y = grid
points_x = (points_x - self.omega[0]) / self.h[0]
points_y = (points_y - self.omega[2]) / self.h[1]

# Interpolate
C = naturalSpline3()

j = np.arange(-3, C.shape[1]-3) + .5
k = np.arange(-3, C.shape[0]-3) + .5

shift_x = points_x[:, :, np.newaxis, np.newaxis] - j[np.newaxis, np.newaxis, :, np.newaxis]
shift_y = points_y[:, :, np.newaxis, np.newaxis] - k[np.newaxis, np.newaxis, np.newaxis, :]

splines_x = Bspline3(shift_x)
splines_y = Bspline3(shift_y)

f = splines_x[0] * splines_y[0]
s = np.einsum('ijkl, kl->ij', f, C)

g = np.stack((splines_x[1] * splines_y[0],
              splines_x[0] * splines_y[1]), axis=0)
ds = np.einsum('dijkl, kl->dij', g, C)
return s, ds

def SSD(self, dataR, dataT):
    hd = np.prod(self.h)

    r = dataR - dataT
    ssd = .5 * hd * np.linalg.norm(r, ord='fro')**2
    dssd = r * hd
    return ssd, dssd

```

```

def plot(self, dataR, dataT, phi, w, state=''):
    fig = plt.figure()
    fig.set_size_inches(12, 8)
    fig.tight_layout(pad=4.0)

    fig.suptitle(f'{state}\nRotation um {np.round(w, 2)[0]} Rad '
                f'und Translation um {tuple(np.round(w, 2)[1:])}', fontsize=16)

    ax1 = fig.add_subplot(231)
    im1 = ax1.imshow(self.ref_image, vmin=0, vmax=255, cmap='bone')
    ax1.axis('image')
    ax1.set_title('R')

    ax2 = fig.add_subplot(232)
    ax2.imshow(self.tmp_image, vmin=0, vmax=255, cmap='bone')
    ax2.set_title(r'T und $\varphi$')

    step = self.m[0] // 10
    step_r = int((self.m[0] % 10) // 4)
    grid_plot = [x.reshape(self.m) for x in phi[:, :-1]]
    grid_plot = [x[step_r:-1:step, step_r:-1:step].flatten() for x in grid_plot]
    ax2.scatter(*grid_plot, c='w', marker='.')

    ax2.set_xlim([0, self.m[0]-1])
    ax2.set_ylim([0, self.m[1]-1][::-1])

    ax3 = fig.add_subplot(233)
    ax3.imshow(self.ref_image - self.tmp_image, vmin=-255, vmax=255, cmap='bwr')
    ax3.set_title(rf'$diff(R, T)$, SSD = {round(self.SSD(self.ref_image, self.tmp_image)[0], 2)}$')

    ax5 = fig.add_subplot(235)
    ax5.imshow(dataT, vmin=0, vmax=255, cmap='bone')
    ax5.set_title(rf'$T \circ \varphi$: Transformiertes T')

    ax6 = fig.add_subplot(236)
    diff = ax6.imshow(dataR - dataT, vmin=-255, vmax=255, cmap='bwr')
    ax6.set_title(rf'$diff(R, T \circ \varphi)$, SSD = {round(self.SSD(dataR, dataT)[0], 2)}$')

    cax1 = plt.axes([.03, 0.11, .02, .77])
    fig.colorbar(im1, cax=cax1, aspect=1, pad=.1)
    cax2 = plt.axes([.92, 0.11, .02, .77])
    fig.colorbar(diff, cax=cax2, aspect=1, pad=.1)

    plt.show()

def objective_fun_ir(self, w, plot, title):
    phi, dphi = self.deform_grid(self.tmp_grid, w)

    dataR, ddataR = self.interpolate(self.ref_image, [x.reshape(*self.m) for x in self.ref_grid])
    dataT, ddataT = self.interpolate(self.tmp_image, [x.reshape(*self.m) for x in phi])

    ssd, dssd = self.SSD(dataR, dataT)

    val = ssd
    grad = dphi * np.reshape(-ddataT * dssd[np.newaxis, :, :], (1, 2, np.prod(ddataT.shape[1:])))

```

```

        grad = np.sum(grad, axis=(1, 2))
        if plot:
            self.plot(dataR, dataT, phi, w, state=title)
        return val, grad

if __name__ == '__main__':
    rr = RigidRegistration()

    # TODO: Aufgabenteil 1. Verschiedene w0 testen und sich mit objective_fun_ir vertraut machen.
    # Achtung: Drehwinkel in Radian und nicht in Grad.
    test_w0 = {
        0: np.array([0, 0, 0]), # weder Rotation noch Translation
        1: np.array([0.4, 0, 0]), # Rotation
        2: np.array([-0.4, -1, -1]), # Rotation
        3: np.array([0, 5, 0]), # Translation x-Richtung
        4: np.array([0, 0, 2]), # Translation y-Richtung
        5: np.array([-0.51532779, 0.00690073, 2.00917066])
    }

    # Hier testen:
    for i in range(6):
        _, _ = rr.objective_fun_ir(test_w0[i], plot=True, title='Beispielaufruf der Zielfunktion')

    # TODO: Aufgabenteil 1. Für Verwendung des Quasi-Newton-Verfahrens geeignete Funktion f erstellen.
    # Hinweis: Der "*" -Operator zur Auflösung von Tupeln könnte hilfreich sein.
    # BEGIN SOLUTION
    f = lambda w: (*rr.objective_fun_ir(w, plot=False, title=None), None)
    # END SOLUTION

    # TODO: Aufgabenteil 3. Parameter der Bildregistrierung für verschiedene Startwerte mit Quasi-Newton optimieren.
    # Hinweis: - Importieren Sie Ihre Quasi-Newton-Implementierung aus A9_2.
    #           - Bildregistrierung ist rechenintensiv, Geduld.
    # BEGIN SOLUTION
    B0 = np.eye(3)
    rho = 1/2
    c1 = 1e-3
    tau = 1e-5
    eps = 1e-10
    kmax = 100
    w0 = np.array([0, 0, 0])
    w0_self = np.array([-0.4, 0, 2])

    rr.objective_fun_ir(w0_self, plot=True, title='Vor der Optimierung mit w0 = [-0.3, 1, 2]')
    log1 = quasi_newton(f = f, x0 = w0_self, B0 = B0, rho = rho, c1 = c1, eps = eps, tau = tau, kmax = kmax)
    rr.objective_fun_ir(log1['xqn'], plot=True, title='Nach der Optimierung mit w0 = [-0.3, 1, 2]')

    rr.objective_fun_ir(w0, plot=True, title='Vor der Optimierung mit w0 = [0, 0, 0]')
    log2 = quasi_newton(f = f, x0 = w0, B0 = B0, rho = rho, c1 = c1, eps = eps, tau = tau, kmax = kmax, plot=True)
    rr.objective_fun_ir(log2['xqn'], plot=True, title='Nach der Optimierung mit w0 = [0, 0, 0]')

    # END SOLUTION

    # TODO: Aufgabenteil 3. Diskussion.
    # BEGIN SOLUTION
    print("Verändert man das Tau und das Epsilon so stellt man fest, dass z.T bessere Ergebnisse mit größerem Tau und kleinerem Epsilon erzielt werden können.")

```

```
"Tau und Epsilon erreicht werden.\n"
"Deutlich wichtiger für das Finden eines guten Minimierers ist der Startwert.")
# END SOLUTION
```

#TERMINAL OUTPUT:

Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.
Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.

Abbruch nach GMW 3 erfüllt

Verändert man das Tau und das Epsilon so stellt man fest, dass z.T. bessere Ergebnisse mit größeren Tau und Epsilon erreicht werden.

Deutlich wichtiger für das Finden eines guten Minimierers ist der Startwert.

#PLOTS:

Anmerkung: Das Programm braucht bei uns $\sim \frac{3}{4}h$ und das Ergebnis sind garantiert keine Minimize.

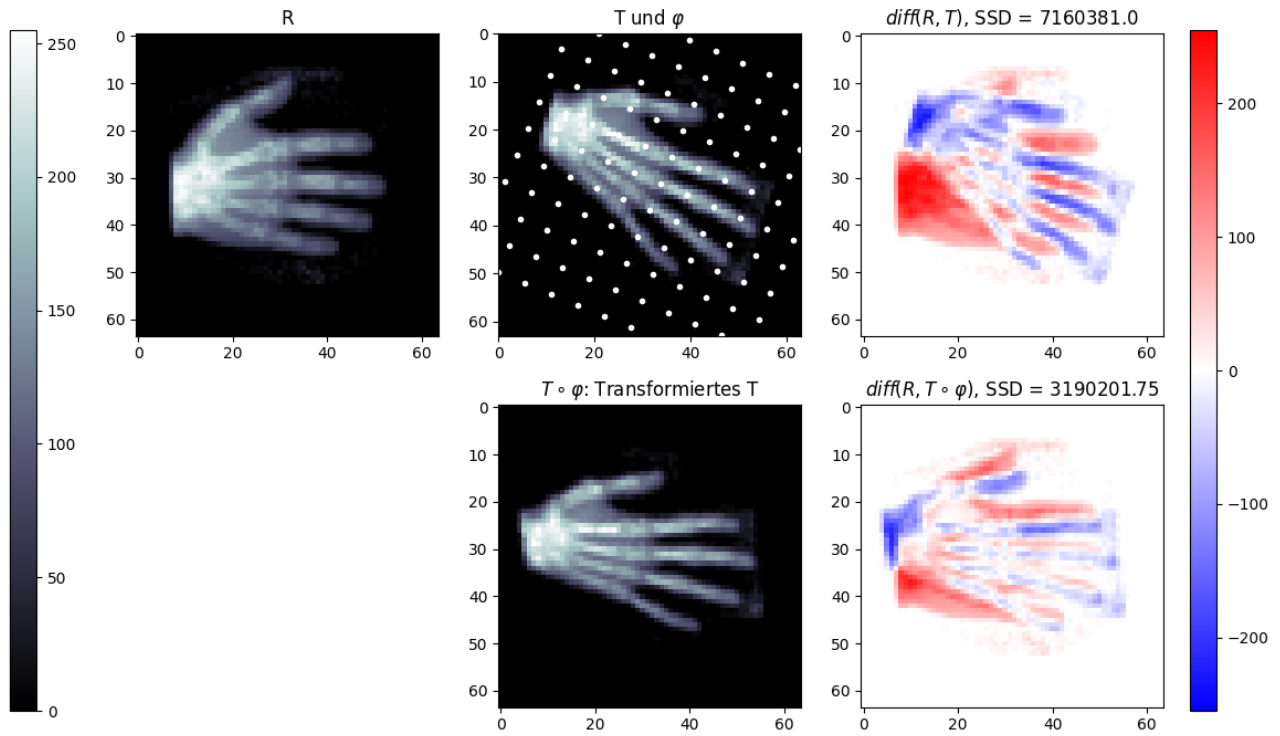
Dabei sind im Iterationsverlauf sehr gute Kandidaten vorhanden (ein solcher ist der in Test-W0 angefügte 5. Punkt)

jedoch scheitert dann die Liniensuche und es kommt zu einem deutlich schlechteren Ergebnis.

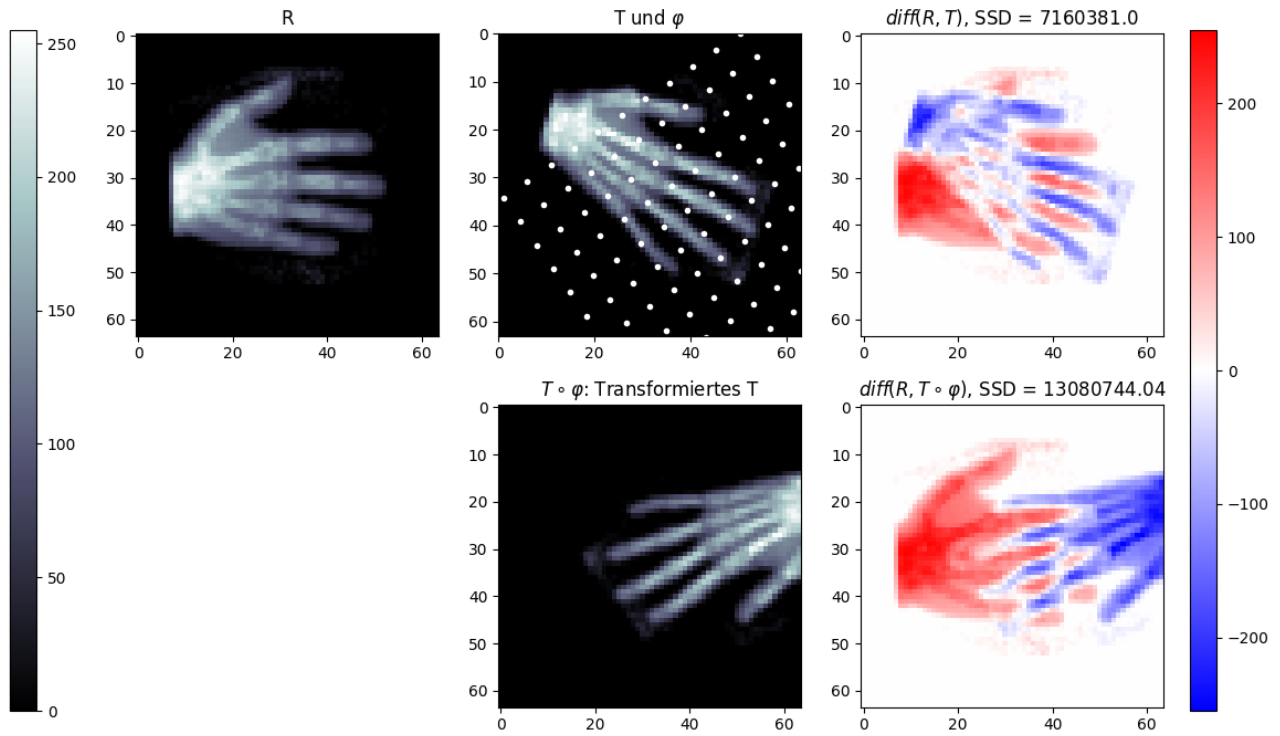
Nicht auszuschließen ist hier natürlich, dass unsere Implementierung von Quasi-Newton einfach nicht gut ist.

Aufgrund der Laufzeit und der $\sim 8h$ Zeitaufwand in diese Aufgabe haben wir uns entschieden, es hier zu belassen.

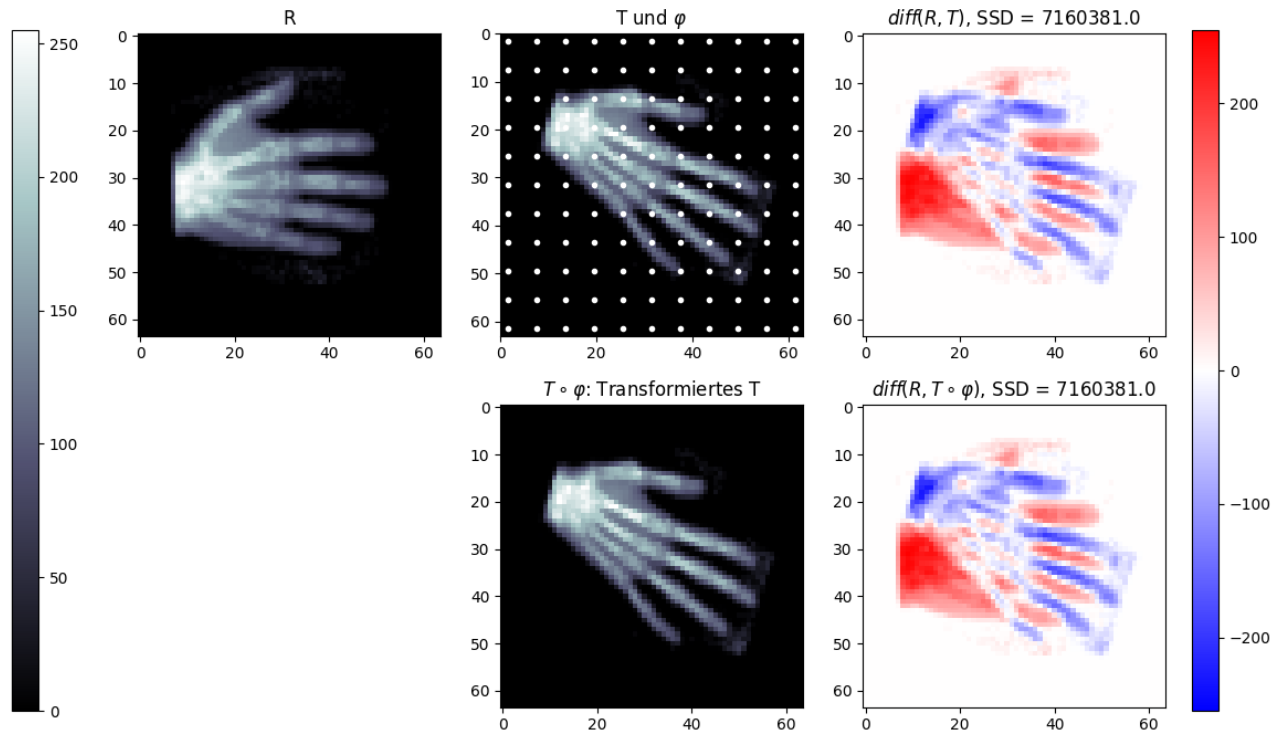
Vor der Optimierung mit $w_0 = [-0.3, 1, 2]$
 Rotation um -0.4 Rad und Translation um $(\text{np.float64}(0.0), \text{np.float64}(2.0))$



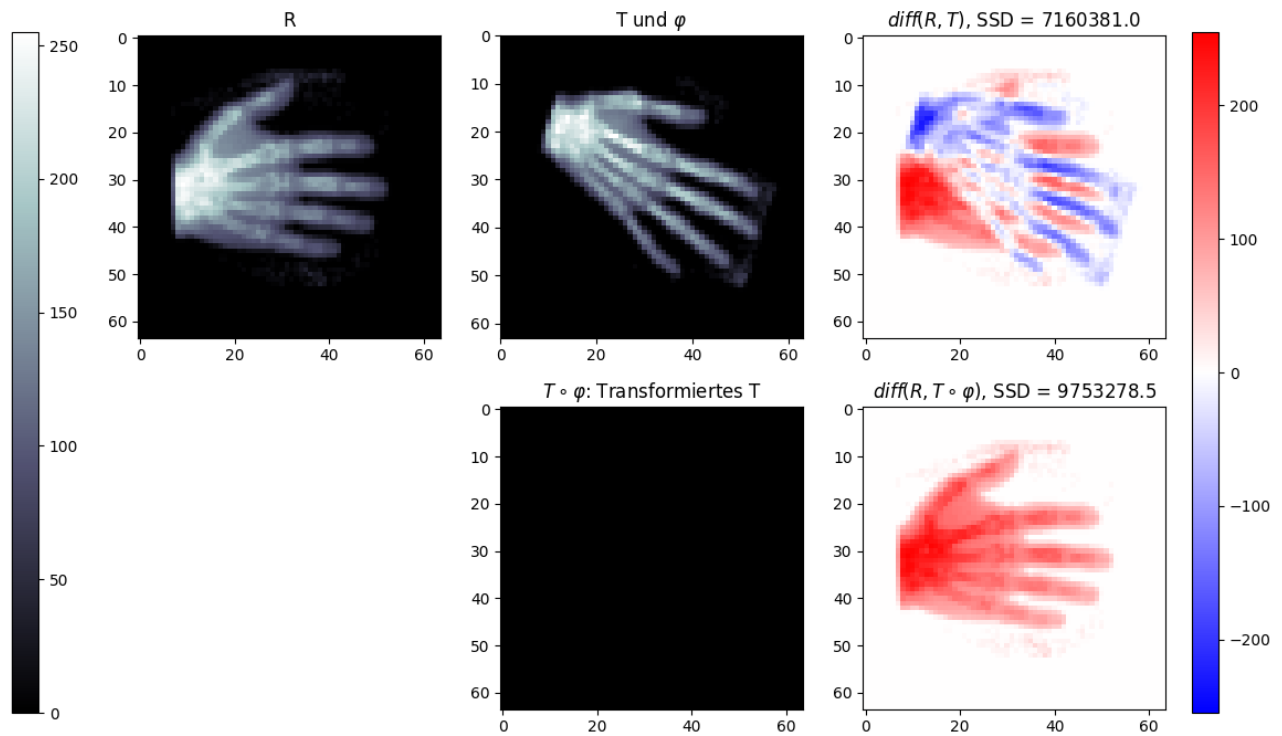
Nach der Optimierung mit $w_0 = [-0.3, 1, 2]$
 Rotation um 8.46 Rad und Translation um $(\text{np.float64}(8.99), \text{np.float64}(10.99))$



Vor der Optimierung mit $w_0 = [0, 0, 0]$
 Rotation um 0 Rad und Translation um $(\text{np.int64}(0), \text{np.int64}(0))$



Nach der Optimierung mit $w_0 = [0, 0, 0]$
 Rotation um 129.72 Rad und Translation um $(\text{np.float64}(141.17), \text{np.float64}(143.53))$



Code/utils.py

```
import matplotlib.pyplot as plt
import numpy as np

# Uebernommen von Blatt 8
def armijo(f, p, x, rho, c1, alpha0=1):
    """
    Armijo Backtracking Liniensuche

    Input:
        f: Funktion, wobei  $f(x) = (val, grad, hess)$ ,
        p: Vektor (Abstiegsrichtung)
        x: Vektor (Iterierte)
        rho: Skalar (Reduktionsfaktor für alpha)
        c1: Skalar (Parameter für die Armijo-Bedingung)
        alpha0: Skalar (Initial getesteter Startwert)

    Output:
        alpha: Skalar (gesuchte Schrittweite)
    """

    # Initialisierung
    alpha = alpha0

    # Vorberechnung reduziert Laufzeit in der Schleife
    valk, gradk, _ = f(x)
    descent = gradk @ p

    for lsiter in range(100):
        valkp1, _, _ = f(x + alpha * p)

        # Testen der ``sufficient decrease condition``
        if valkp1 <= valk + c1 * alpha * descent:
            return alpha

        # Reduzieren von alpha
        alpha = rho * alpha

    # Fehlermeldung, falls die Liniensuche nicht auskonvergiert ist
    print("Liniensuche wurde nach 100 Schritten frühzeitig abgebrochen.")
    return alpha

# Uebernommen von Blatt 8
def gill_murray_wright(xk, xkplus1, valk, valkplus1, gradk, k, eps, tau, kmax):
    """
    Abbruchkriterien nach Gill, Murray und Wright

    Input:
        xk: Vektor (aktuelle Iterierte)
        xkp1: Vektor (nächste Iterierte)
        valk: Skalar ( $f(xk)$ )
        valkp1: Skalar ( $f(xkp1)$ )
    """
```

```

gradk: Vektor ( $\nabla f(x_k)$ )
k: Integer (Iteration-Nummer)
tau: Skalar (Toleranz für Kriterium 1)
eps: Skalar (Toleranz für Gradient -- Kriterium 3)
kmax: Integer (maximale Anzahl der Iterationen)

```

Output:

```

test: Boolescher Wert (Ergebnis der Abbruchkriterien nach Gill, Murray, Wright)
"""

```

```

test = False
if (valk - valkplus1 < tau * (1 + np.abs(valk))
    and np.linalg.norm(xkplus1 - xk) < np.sqrt(tau) * (1 + np.linalg.norm(xk))
    and np.linalg.norm(gradk) < np.cbrt(tau) * (1 + np.abs(valk))):
    test = True
    print('\nAbbruch nach GMW 1 erfuehlt')

if np.linalg.norm(gradk) < eps:
    test = True
    print('\nAbbruch nach GMW 3 erfuehlt')

if k == kmax:
    test = True
    print('\nAbbruch nach GMW 3 erfuehlt')

return test

```

Uebernomen von Blatt 8

```
def rosenbrock(x, y):
```

```
"""
```

Implementierung der Rosenbrock-Funktion $f(x, y) = 100(y - x^2)^2 + (1 - x)^2$

Input:

x, y : Skalare bzw. Arrays zur Auswertung der Funktion.

Output:

```

val: Zielfunktionswert(e),
grad: Gradient(en),
hess: Hessematri(x/zen).
"""

```

```

val = 100 * (y - x ** 2) ** 2 + (1 - x) ** 2
grad = np.stack((- 400 * x * (y - x ** 2) - 2 * (1 - x), 200 * (y - x ** 2)), axis=-1)
hess = np.stack((np.stack((-400 * (y - x ** 2) + 800 * x ** 2 + 2, -400 * x), axis=-1),
                 np.stack((-400 * x, 200 * np.ones_like(x)), axis=-1)), axis=1)

return val, grad, hess

```

Anepasst von Blatt 8

```
def plot_iteration_rosenbrock(log, title):
```

```
"""
```

Funktion zur Erstellung von Plots zum Optimierungsverlauf der Rosenbrock-Funktion anhand der Ergebnisse. Im ersten Plot wird der Iterationsverlauf auf einem Surface-Plot dargestellt, in weiteren 3 Konvergenz. Die Funktion liefert fig zurück. Stellen Sie diese dar, indem Sie plt.show() in Ihrer Hauptdatei aufrufen.

Input:

log: Dictionary, das mindestens folgende Einträge enthält:

```
log = {  
    'x0': Vektor,           # Startwert  
    'x_list': Liste,        # Iterierte  
    'val_list': Liste,       # Funktion ausgewertet an den Iterierten  
    'norm_grad_list': Liste, # Norm des Gradienten ausgewertet an dem Iterierten  
}
```

title: Titel des Plots.

Output:

fig: hergestellte Figure mit den Plots

"""

Figure initialisieren

```
fig, ax = plt.subplots(2, 2)  
fig.set_size_inches(9, 6)  
fig.tight_layout(pad=3, h_pad=3)  
fig.suptitle(title)
```

Surface-Plot mit Iterationsverlauf

```
x = np.linspace(-1., 1.5, 651)  
xx, yy = np.meshgrid(x, x)  
zz = rosenbrock(xx.flatten(), yy.flatten())[0].reshape((651, 651))  
CS = ax[0, 0].contour(xx, yy, zz, levels=[2, 25, 75, 150, 300, 500, 700])  
ax[0, 0].clabel(CS, inline=1, fontsize=10)  
ax[0, 0].scatter(*log['x0'], marker='o')  
ax[0, 0].scatter(1, 1, marker='*')  
for k in range(len(log['x_list']) - 1):  
    xk = log['x_list'][k]  
    xkplus1 = log['x_list'][k + 1]  
    ax[0, 0].plot([xk[0], xkplus1[0]], [xk[1], xkplus1[1]])  
ax[0, 0].set_title('Höhenlinien mit Iterationsverlauf')
```

Plotten der Zielfunktionsfehler

```
val_list = log['val_list']  
val_star = 0  
for k, valk in enumerate(val_list):  
    ax[0, 1].scatter(k, valk - val_star)  
ax[0, 1].set_yscale('log')  
ax[0, 1].set_title(r'$f(x^k) - f(x^{\ast})$')
```

Plotten der Gradientennorm

```
norm_grad_list = log['norm_grad_list']  
for k, norm_gradk in enumerate(norm_grad_list):  
    ax[1, 0].scatter(k, norm_gradk)  
ax[1, 0].set_yscale('log')  
ax[1, 0].set_title(r'$||\nabla f(x^k)||$')
```

Plotten der Konvergenz der Iterierten

```
x_list = log['x_list']  
xstar = np.array([1, 1])  
for k, xk in enumerate(x_list[1:]):  
    ax[1, 1].scatter(k, np.linalg.norm(xk - xstar))  
ax[1, 1].set_yscale('log')  
ax[1, 1].set_title(r'$||x^k - x^{\ast}||$')
```

```
return fig
```

```
#TERMINAL OUTPUT:
```

```
#PLOTS:
```